

Assignment 2 - Data Pipeline Creation

Submitted by : Sruthilaya Umasankari Soma Shanmuga Sundaram

Quick Summary of what was implemented:

In this assignment, I built a complete data preprocessing pipeline for a small customer dataset with 10 rows containing country, age, salary, and purchase information.

Q1: I loaded the dataset and handled missing values in Age and Salary using sklearn's SimpleImputer with three strategies mean, median, and most_frequent. I also wrote unit tests to verify edge cases like all-NaN columns.

Q2: I encoded the data for ML one-hot encoded Country into separate binary columns (France/Germany/Spain) and label encoded the target Purchased into 0s and 1s. I verified the encoding was reversible by decoding back to original labels.

Q3: I experimented with different train/test split sizes and found that with only 10 rows, class balance varies heavily by chance. test_size=0.4 was most stable. I also compared three scalers StandardScaler gave mean=0 std=1, MinMaxScaler compressed to 0-1, and RobustScaler handled outliers better using median/IQR.

Q4: I combined all steps into a single sklearn Pipeline that automatically imputes, encodes, and scales raw data without manual intervention. I tested it on unseen data including unknown countries (handled gracefully) and verified the pipeline produces identical results after saving and loading from disk.

(note: Summary written by Llm based on my input, used Llm here to polish the writing only)

Below are the output Screenshots:

Q1. Data Loading and Missing Value Handling

a) From the function load_and_inspect

```

... First 5 rows of the dataset:
      Country  Age  Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0        Yes
2   Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4   Germany  40.0     NaN        Yes

Dataset shape (rows, columns):
(10, 4)

Column data types:
Country      object
Age          float64
Salary       float64
Purchased    object
dtype: object

Missing values % in each column:
      count  percentage
Country      0         0.0
Age          1        10.0
Salary       1        10.0
Purchased    0         0.0
...
25%   35.000000  54000.000000
50%   38.000000  61000.000000
75%   44.000000  72000.000000
max    50.000000  83000.000000

```

b) From the function `handling_missing` we have imputed using mean value

```

Index(['Age', 'Salary'], dtype='object')
(10, 2)

Dataset after handling missing values:
      Country  Age  Salary Purchased
0   France  44.0  72000.000000        No
1    Spain  27.0  48000.000000        Yes
2   Germany  30.0  54000.000000        No
3    Spain  38.0  61000.000000        No
4   Germany  40.0  63777.777778        Yes

```

c) From the unit tests, every assertion passed

```

...
Test DataFrame:
   A  B  C
0 NaN  1  4.0
1 NaN  2  NaN
2 NaN  3  6.0
Index(['A', 'B', 'C'], dtype='object')
(3, 2)

```

Q2. Encoding Pipeline with Validation

a) From function encode features the output of one hot encoding and label encoding

```
One-Hot Encoded features (X):
[[1.00000000e+00 0.00000000e+00 0.00000000e+00 4.40000000e+01
 7.20000000e+04]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00 2.70000000e+01
 4.80000000e+04]
 [0.00000000e+00 1.00000000e+00 0.00000000e+00 3.00000000e+01
 5.40000000e+04]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00 3.80000000e+01
 6.10000000e+04]
 [0.00000000e+00 1.00000000e+00 0.00000000e+00 4.00000000e+01
 6.37777778e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 3.50000000e+01
 5.80000000e+04]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00 3.87777778e+01
 5.20000000e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 4.80000000e+01
 7.90000000e+04]
 [0.00000000e+00 1.00000000e+00 0.00000000e+00 5.00000000e+01
 8.30000000e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 3.70000000e+01
 6.70000000e+04]]

Label Encoded target (y):
[0 1 0 0 1 1 0 1 0 1]
```

b) Features names after encoding

```
One-Hot Encoded feature names:
encoder__Country_France encoder__Country_Germany encoder__Country_Spain \
0 1.0 0.0 0.0
1 0.0 0.0 1.0
2 0.0 1.0 0.0
3 0.0 0.0 1.0
4 0.0 1.0 0.0
5 1.0 0.0 0.0
6 0.0 0.0 1.0
7 1.0 0.0 0.0
8 0.0 1.0 0.0
9 1.0 0.0 0.0

remainder__Age remainder__Salary
0 44.000000 72000.000000
1 27.000000 48000.000000
2 30.000000 54000.000000
3 38.000000 61000.000000
4 40.000000 63777.777778
5 35.000000 58000.000000
6 38.777778 52000.000000
7 48.000000 79000.000000
8 50.000000 83000.000000
9 37.000000 67000.000000
```

c) Result of Decoding

```
#decoding the encoded target variable to check if it matches the original value.
def decode_predictions(y_encoded, le):
    return le.inverse_transform(y_encoded)
decoded = decode_predictions(y, le)
print(decoded)
assert list(decoded) == list(df['Purchased']), "Decoded values don't match originals"
```

✓ 0.0s Python

['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']

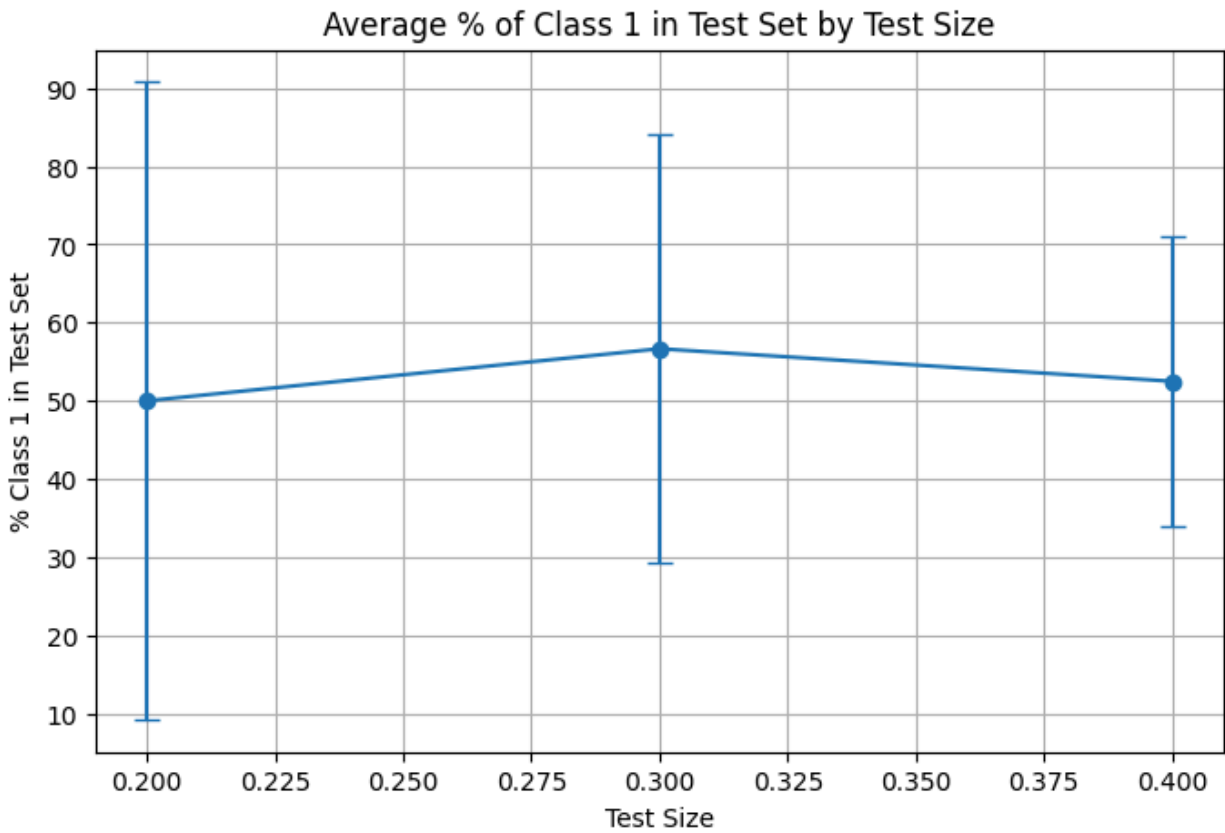
Q3. Train and Test Split Analysis and Scalling

a) From function split_experiment

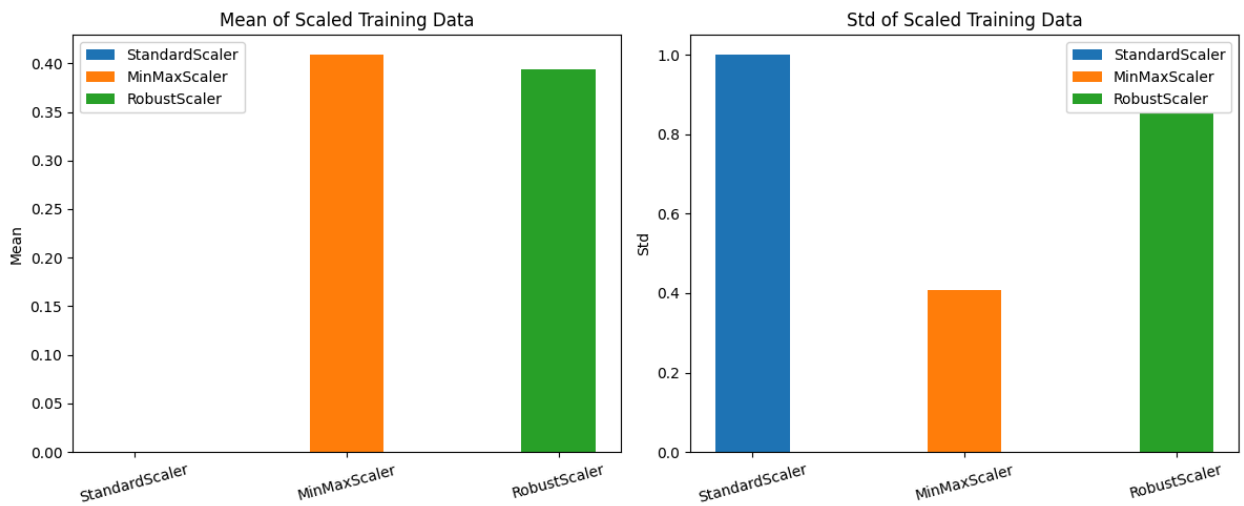
```
...
Experiment results:
   test_size  random_state  train_class_0  train_class_1  test_class_0  \
0         0.2           0           3           5           2
1         0.2           1           4           4           1
2         0.2           2           5           3           0
3         0.2           3           5           3           0
4         0.2           4           3           5           2
5         0.2           5           5           3           0
6         0.2           6           4           4           1
7         0.2           7           4           4           1
8         0.2           8           3           5           2
9         0.2           9           4           4           1
10        0.3           0           3           4           2
11        0.3           1           3           4           2
12        0.3           2           5           2           0
13        0.3           3           5           2           0
14        0.3           4           3           4           2
15        0.3           5           4           3           1
16        0.3           6           4           3           1
17        0.3           7           3           4           2
18        0.3           8           3           4           2
19        0.3           9           4           3           1
20        0.4           0           3           3           2
21        0.4           1           3           3           2
...
26         2
27         1
28         1
29         2

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

b) From plot comparing the chosen test size results this shows 0.4 has better class distribution



c) From compare scaler function



Q4. End to End Resuable Pipeline

a) From preprocessor step

```
...   num_Age  num_Salary  cat_Country_France  cat_Country_Germany  \
0   0.462364    0.421194                1.0                0.0
1  -0.294232   -0.475542                0.0                0.0
2  -1.681325   -1.535320                0.0                0.0
3   0.966762    0.991844                1.0                0.0
4   1.218961    1.317930                0.0                1.0
5  -0.672530   -0.720106                1.0                0.0

   cat_Country_Spain
0                0.0
1                1.0
2                1.0
3                0.0
4                0.0
5                0.0

   num_Age  num_Salary  cat_Country_France  cat_Country_Germany  \
0  -1.303027   -1.046192                0.0                1.0
1  -0.420331    0.013587                1.0                0.0
2  -0.196155   -1.209235                0.0                0.0
3  -0.042033   -0.249093                0.0                1.0

   cat_Country_Spain
0                0.0
1                0.0
2                1.0
3                0.0
```

b) From pipeline test step

```
new_data = pd.DataFrame({
    'Country': ['Italy', 'Germany', 'France'],
    'Age': [25.0, np.nan, 55.0],
    'Salary': [60000.0, 70000.0, 80000.0]
})
test_pipeline_on_new_data(preprocessor, new_data)
```

9] ✓ 0.0s

Row 0 fails: Found unknown categories ['Italy'] in column 0 during transform skip
Row 1 transformed successfully: [[0. 0.25815122 0. 1. 0. 0.]]
Row 2 transformed successfully: [[1.84945795 1.0733656 1. 0. 0. 0.]]

c) From saving and loading pipeline

... Pipeline saved, loaded successfully and outputs match

